

SRCS : Systèmes Répartis Client/Serveur

TD 5 : Protocoles de sécurité

Objectifs pédagogiques

- Messages chiffrés
- Authentification sur un réseau
- Clés secrètes et clés publiques

Exercice 1 – Protocole Kerberos



Kerberos est un protocole distribué d'authentification élaboré par le MIT et fonctionne à l'aide de clés secrètes. Il fait partie des protocoles d'authentification les plus implantés. Le but de ce protocole est d'assurer que seuls des utilisateurs autorisés puissent utiliser des stations de travail distantes ou des services distants (ex : serveur d'impression). Dans sa version la plus simple le protocole considère trois type d'entités :

- **Les clients** : les processus qui souhaitent accéder aux différents services offerts par le système
- **Les serveurs** : les processus qui offrent des services distants
- **le Key Distribution Center (KDC)** : un processus serveur propre au protocole Kerberos qui stocke et génère l'ensemble des clés secrètes des différentes entités.

Question 1

Rappeler les caractéristiques d'un chiffrement à clé secrète.

Pour qu'un client puisse utiliser un serveur, le protocole Kerberos se base sur une distribution de clés de sessions en respectant les étapes suivantes :

1. Le client envoie au KDC une demande d'utilisation du serveur.
2. Le KDC s'assure que le client est bien autorisé à utiliser le service demandé et renvoie une réponse en conséquence. En cas de succès, le message de réponse contient un ticket d'utilisation du service (contenant entre autres la clé de session générée aléatoirement) autorisant le client à contacter le serveur demandé. Sinon, en cas d'échec, c'est un message d'erreur qui est envoyé.
3. Le client envoie sa requête au serveur en renseignant le ticket

Comme dans tout système d'authentification, tout client doit convenir au préalable avec l'administrateur du système d'un identifiant et d'une clé secrète (comme par exemple un mot de passe) connu seulement par le KDC et le client en question. Il est également nécessaire que toute entité serveur puisse être enregistrée auprès du KDC. Ainsi chaque service possède également une clé secrète qu'il partage avec le KDC.

Question 2

Quelle hypothèse est nécessaire sur le KDC pour que le protocole fonctionne ?

Question 3

Quels sont les contenus de chacun de ces messages ? Quelles sont les informations transmises qui doivent être chiffrées et par quelle clé ? Le but final étant que le serveur puisse s'assurer que le client qui le contacte est bien celui qu'il prétend être.

Question 4

Montrer que le serveur puisse s'assurer que le client est bien ce qu'il prétend être quand il le contacte.

Question 5

Quelle hypothèse est-il nécessaire de faire sur les horloges des différentes entités ?

Question 6

Discuter le fait de se baser sur UDP ou sur TCP.

Question 7

Que faudrait-il faire pour que le client s'assure qu'il adresse bien au bon serveur ?

Question 8

Quel est l'inconvénient de n'avoir qu'un seul serveur pour le KDC ? Qu'apporterai un deuxième serveur ?

Exercice 2 – Problème de la notarisation

On souhaite réaliser un service ayant pour objectif la non répudiation de transactions entre deux sites A et B : A fait office de client et B fait office de serveur.

Question 1

Rappeler la définition du problème de non-répudiation.

Pour réaliser ce service nous avons besoin d'un troisième site N qui fait office de notaire. On propose le protocole suivant en 6 étapes à base de clés secrètes. On suppose que tous les sites i partagent une clé secrète K_i avec le notaire mais que le notaire ne partage sa clé secrète avec personne.

- **Étape 1** : le site A envoie un message à N et la chiffrant avec la clé K_A . Le message contient 3 informations : l'identifiant de l'émetteur de la transaction (ici A), le destinataire de la transaction (ici B) et l'objet de la transaction (que l'on notera O). On a donc

$$A \dashrightarrow K_A(A, B, O) \dashrightarrow N$$

- **Étape 2** : Le notaire N déchiffre le message avec K_A , date la transaction avec une estampille temporelle T et répond à A en chiffrant ces informations avec K_N . On a donc

$$N \dashrightarrow K_N(A, B, O, T) \dashrightarrow A$$

- **Étape 3** : le site A transmet le message reçu à B . On a donc

$$A \dashrightarrow K_N(A, B, O, T) \dashrightarrow B$$

- **Étape 4** : le site B transmet le message reçu à N . On a donc

$$B \dashrightarrow K_N(A, B, O, T) \dashrightarrow N$$

- **Étape 5** : le notaire N déchiffre le message et renvoie le contenu à B en le chiffrant avec K_B . On a donc

$$N \dashrightarrow K_B(A, B, O, T) \dashrightarrow B$$

- **Étape 6** : B déchiffre le message, exécute la transaction et acquitte au notaire N la transaction en le chiffrant avec K_B . On a donc

$$B \dashrightarrow K_B\left(\text{Ack}(A, B, O, T)\right) \dashrightarrow N$$

Question 2

Pour chacune des étapes citées ci-dessus, préciser les propriétés de sécurités obtenues (confidentialité, authentification, intégrité, non répudiation) vis à vis de tous les participants.

Question 3

Proposer une version du protocole en utilisant des clés publiques en respectant exactement les mêmes propriétés.

Question 4

On suppose que l'utilisateur client A dispose d'un délai d_{max} pour signaler la perte de sa clé. Toute perte non signalée dans le délai d_{max} ne peut-être tenue à charge contre l'utilisateur. Toute perte non signalée après le délai d_{max} entraîne la responsabilité totale du client en cas de contestation. Que pourrait faire le notaire pour que le serveur B et le notaire ne soient jamais piégés par un client malveillant qui déclare la perte de sa clé après avoir donné un ordre et avant d_{max} . Dans quelles applications une telle stratégie est-elle utilisable ? Dans quelles applications est-elle inutilisable ?

Question 5

Le site B peut essayer de dénier avoir reçu la requête (s'il ne lui convient pas) en n'émettant pas l'acquiescement final et en prétendant ensuite qu'il n'a jamais reçu la transaction interprétable ou que son ordinateur est tombé en panne à ce moment. Comment le notaire peut-il contrer cette attitude ?